

Week 9 Progress Report

Project: mfja_3rd_floor_gz
Subsystem: Room 315 kinematic shuttle simulation
Environment: ROS 2, Gazebo, rail-specific right/left shuttle nodes
Date: 05 May 2026

This report summarizes the current technical state of the Room 315 shuttle simulation after the Week 9 updates. The work focuses on deterministic kinematic motion, explicit routing through the rail topology, visual switch synchronization in Gazebo, and maintainable ROS 2 interfaces for right and left rail operation.

1. Summary of Work

The shuttle is controlled as a kinematic body following calibrated rail geometry rather than wheel-contact physics. The node computes the shuttle position along a directed segment network, resolves switch-dependent transitions, transforms the resulting pose into the Gazebo world frame, and publishes runtime state and sensor information through ROS 2 topics.

2. Main Week 9 Updates

- Separated right and left rail operation through explicit namespaces: `/room_315_right` and `/room_315_left`.
- Kept visual switch updates active while avoiding unnecessary Gazebo world pauses during switch changes.
- Simplified switch command usage by documenting public labels `A1` to `A4` instead of visual-only labels such as `A1R` or `A1L`.
- Added a routing-topology table that describes the next segment and the switch condition required at each transition.
- Documented the pose calibration chain from rail coordinates to Gazebo world coordinates.

3. Runtime Interfaces

Function	Right rail	Left rail
State output	<code>/room_315_right/shuttle/state</code>	<code>/room_315_left/shuttle/state</code>
Shuttle command	<code>/room_315_right/shuttle/control_cmd</code>	<code>/room_315_left/shuttle/control_cmd</code>
Switch command	<code>/room_315_right/switch_states</code>	<code>/room_315_left/switch_states</code>
Stopper command	<code>/room_315_right/stopper_states</code>	<code>/room_315_left/stopper_states</code>
Position sensors	<code>/room_315_right/sensors/position</code>	<code>/room_315_left/sensors/position</code>
Switch approach sensors	<code>/room_315_right/sensors/switch_approach</code>	<code>/room_315_left/sensors/switch_approach</code>

4. Representative Commands

```
ros2 launch mfja_robot_control_config room_315_dual_kinematic_shuttles.launch.py \
  gazebo_world_name:=room_315_only \
  enable_right:=true \
  enable_left:=true \
  right_start_slot:=2 \
  left_start_slot:=1 \
  speed:=0.2

ros2 topic pub --once /room_315_right/switch_states std_msgs/msg/String "{data: 'A1=EXTERIOR'}"
ros2 topic pub --once /room_315_left/switch_states std_msgs/msg/String "{data: 'A2=INTERIOR'}"
```

5. Software Structure

File	Role
room_315_kinematic_shuttle.py	Contains the rail geometry model, arc-length sampling, segment transition logic, and KinematicShuttleCore .
room_315_kinematic_shuttle_node.py	Provides the ROS 2 node, publishers, subscribers, Gazebo service calls, runtime shuttle management, sensors, stoppers, and pose calibration.
rail_network_right.yaml	Defines the right rail topology, segments, switches, sensors, stoppers, and routing table.
rail_network_left.yaml	Defines the left rail topology and rail-specific switch mapping.
conveyor_loop_mode_controller.py	Applies visual switch commands to Gazebo switch models and republishes the current visual switch state.

6. Switch Command Routing

A switch command is first interpreted by the rail-specific shuttle node as a logical route command. The same node then publishes a visual command to the conveyor switch controller so that Gazebo remains consistent with the route state.

Step	Source / node	Topic	Result
1	Operator or supervisor	/room_315_*/switch_states	Publishes a command such as A2=INTERIOR .
2	room_315_kinematic_shuttle_node.py	Internal route state	Updates the logical switch state used by route resolution.
3	room_315_kinematic_shuttle_node.py	/mfja/conveyor/switch_cmd	Publishes the corresponding visual switch command.
4	conveyor_loop_mode_controller.py	/mfja/conveyor/switch_states	Applies the visual update and publishes the resulting switch state.

7. Routing Topology

The routing table defines the next segment after each segment end. G corresponds to EXTERIOR , and S corresponds to INTERIOR . If a guard condition is not satisfied, the shuttle enters FALLING , indicating that no valid successor exists for the current switch configuration.

Current segment	Type	Right rail condition	Left rail condition
A23	switch_select	A3=G → A3G A3=S → A3S	A1=G → A3G A1=S → A3S
A3G , A3S	fixed	A34E / A34I	A34E / A34I
A34E	switch_guard	A4=G → A4G A4=S → FALLING	A2=G → A4G A2=S → FALLING
A34I	switch_guard	A4=G → FALLING A4=S → A4S	A2=G → FALLING A2=S → A4S
A14	switch_select	A1=G → A1G A1=S → A1S	A3=G → A1G A3=S → A1S
A12E	switch_guard	A2=G → A2G A2=S → FALLING	A4=G → A2G A4=S → FALLING
A12I	switch_guard	A2=G → FALLING A2=S → A2S	A4=G → FALLING A4=S → A2S
A2G , A2S	fixed	A23	A23

8. Kinematic Motion Model

The shuttle state is represented by the active segment, the arc-length coordinate `s`, the speed, and the discrete motion mode. On each timer tick, the node advances `s` by `speed * dt`, samples the current segment geometry, resolves transitions at segment boundaries, and publishes the resulting pose.

```
state = (current_segment, s, speed, mode)
s_next = s_current + speed * dt
```

The current implementation supports both polyline sampling and cubic Hermite interpolation. Hermite interpolation is used to reduce discontinuity at CSV points and to obtain a smoother tangent direction for the shuttle yaw angle.

```
yaw = atan2(tangent_y, tangent_x)
```

9. Gazebo Pose Calibration

Rail coordinates and Gazebo world coordinates are treated as separate frames. The shuttle pose computed from the rail geometry is transformed before it is applied to the Gazebo model.

```
[x_b]   [a b] [x] + [tx]
[y_b] = [c d] [y]   [ty]

x_s = origin_x + (x_b - origin_x) * scale_x
y_s = origin_y + (y_b - origin_y) * scale_y

[x_r, y_r] = planar_rotation(x_s, y_s, rotation_deg, rotation_origin)

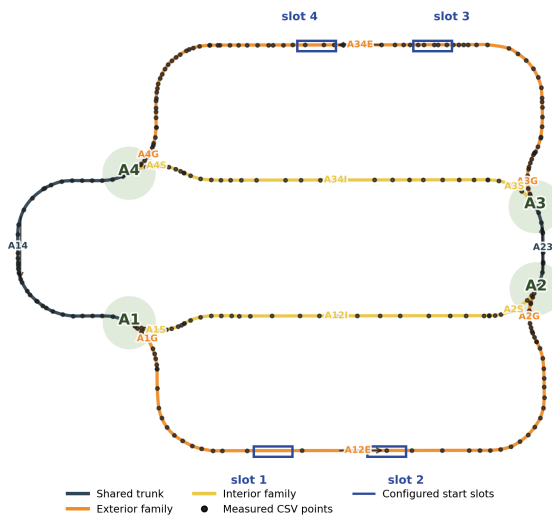
x_gz = x_r + offset_x
y_gz = y_r + offset_y
z_gz = z   + z_offset
```

Term	Meaning
<code>x, y, z</code>	Raw coordinates from the kinematic rail path.
<code>a, b, c, d, tx, ty</code>	Primary affine calibration from rail coordinates toward the Gazebo coordinate frame.
<code>scale_x, scale_y</code>	Residual scale correction applied after the affine transform.
<code>rotation_deg</code>	Residual planar rotation correction.
<code>offset_x, offset_y, z_offset</code>	Final translational correction for visual alignment.

The affine coefficients are obtained by fitting corresponding points between the rail-coordinate dataset and Gazebo world coordinates. The additional scale, rotation, and offset parameters are kept separate to allow fine correction without recomputing the full affine transform.

10. Current Visual Outputs

A. Raw CSV points and cubic Hermite path



B. Sensors, stoppers, and operational points

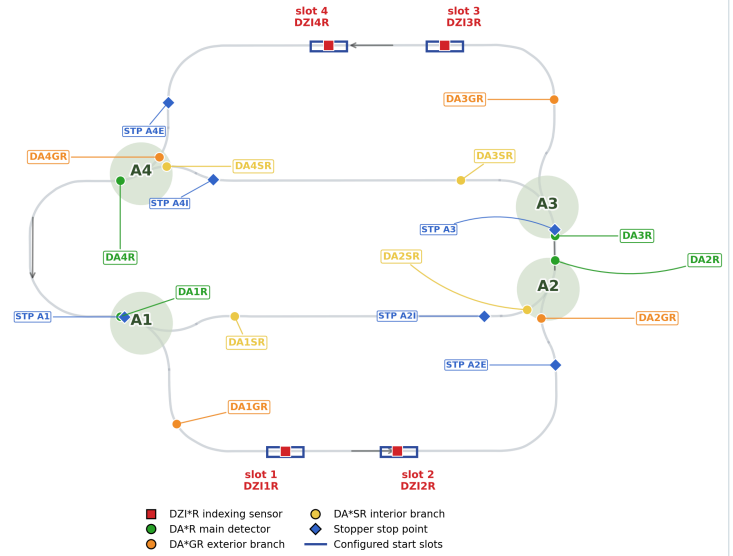
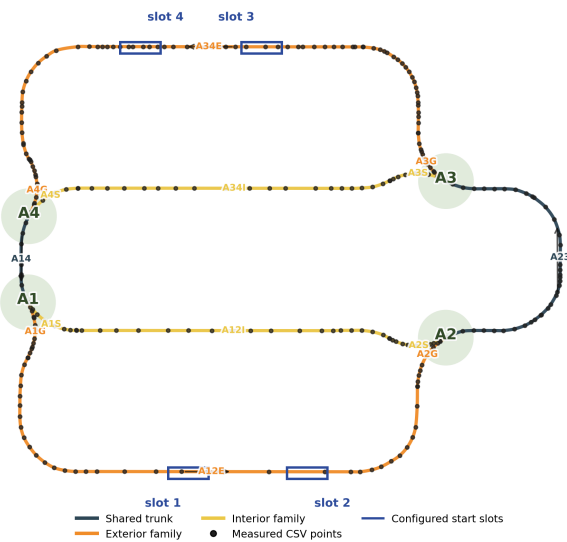


Figure 1. Right rail segment and switch summary.

Room 315 left-rail report figure: geometry, Hermite interpolation, and sensors

A. Raw CSV points and cubic Hermite path



B. Sensors, stoppers, and operational points

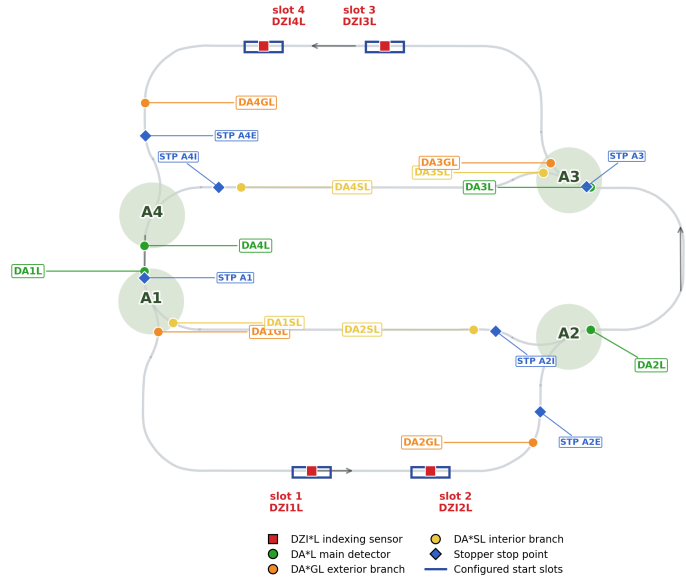


Figure 2. Left rail segment and switch summary.

11. Validation Status

Validation area	Status / purpose
Network topology	Segment references, fixed transitions, switch selectors, and guard transitions are defined explicitly in the rail network YAML files.
Runtime topics	State, position-sensor, switch-approach, and command topics are separated by rail namespace.
Gazebo switch synchronization	Logical switch commands are forwarded to the visual switch controller through <code>/mfja/conveyor/switch_cmd</code> .
Pose alignment	Affine and residual calibration parameters align the kinematic rail path with Gazebo model coordinates.

12. Conclusion

The Week 9 implementation provides a deterministic foundation for Room 315 shuttle simulation. The system separates topology, kinematic path sampling, ROS 2 runtime interfaces, Gazebo visualization, and pose calibration. This structure supports controlled experimentation and further validation of multi-shuttle behavior on the right and left rail networks.